



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

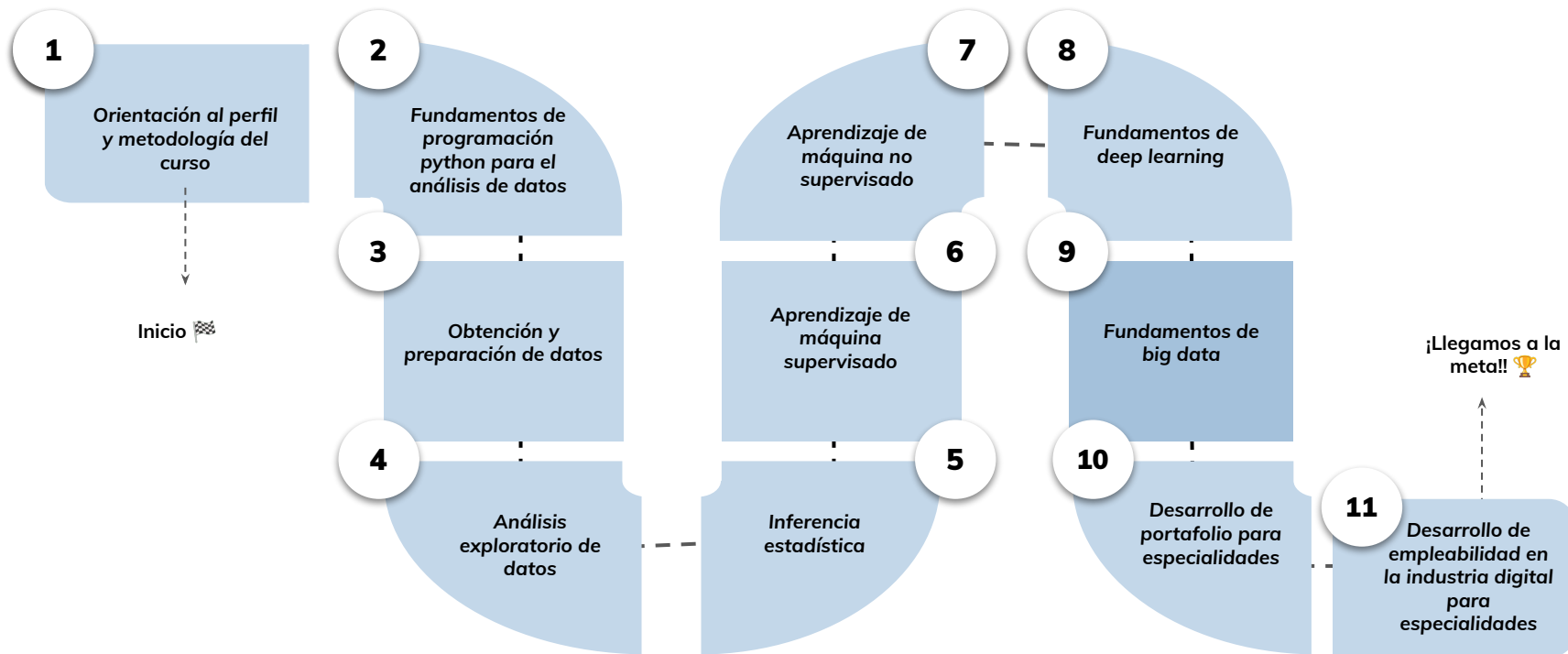


› Introducción a machine learning escalable- Parte I

Aprendizaje Esperado 5: Elaborar un modelo de machine learning (aprendizaje de máquina) utilizando mllib de apache spark para resolver un problema de grandes volúmenes de datos.

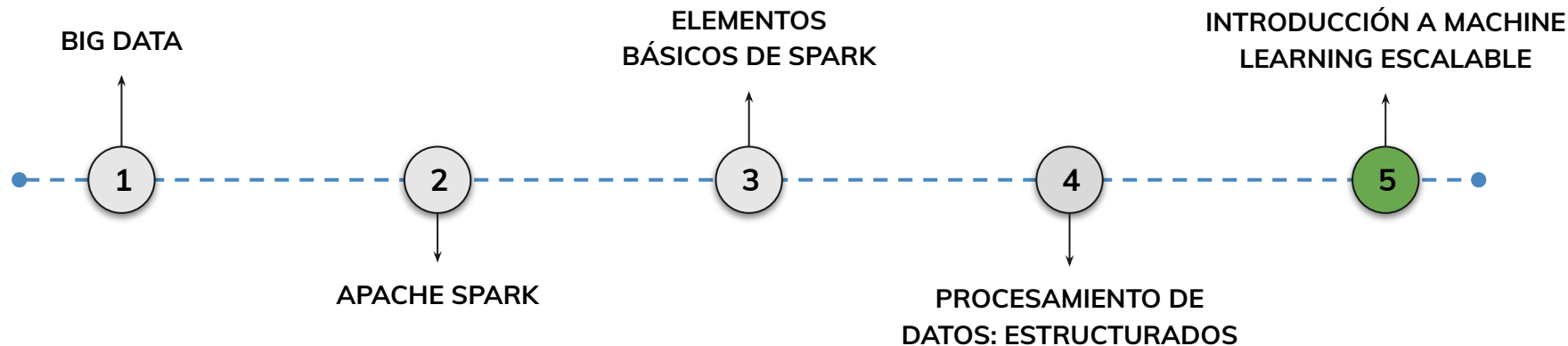
Hoja de ruta

¿Cuáles skills conforman el programa? Fundamentos de Ciencia de Datos



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Fundamentos de MLlib y su ecosistema en Spark

Introduciremos MLlib, la biblioteca de Machine Learning de Apache Spark, explorando sus características, estructuras de datos y algoritmos, con foco en cómo aprovecharla para grandes volúmenes de datos.

Conceptos y características de MLlib

Estructuras de datos y herramientas en MLlib

¿Qué es MLlib y por qué es escalable?

Integración con el ecosistema Spark.

DataFrames, vectores y matrices.

Pipelines de Machine Learning.

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender qué es MLlib y su rol en el procesamiento distribuido.
- Conocer su integración con otros módulos de Spark.
- Identificar algoritmos disponibles y casos de uso.
- Manejar las estructuras de datos utilizadas por MLlib.
- Reconocer herramientas para transformar, evaluar y optimizar modelos.

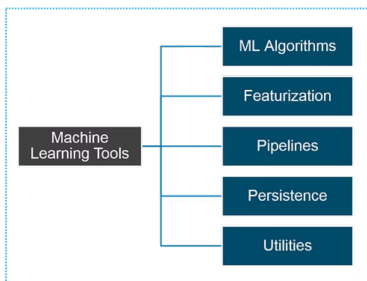
Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Realizamos consultas avanzadas y optimizaciones con Spark SQL.
- Aplicamos funciones de agregación y uniones de DataFrames.
- Usamos selectExpr para ejecutar expresiones SQL directamente sobre DataFrames.
- Analizamos cómo Catalyst y Tungsten optimizan las consultas.

Introducción a machine learning escalable



Apache Spark MLlib: Machine Learning Escalable

En la actualidad, el volumen de datos que manejan las organizaciones ha crecido de forma exponencial, haciendo que los enfoques tradicionales de análisis y modelado de datos resulten insuficientes para procesar información a gran escala. Ante este desafío, surgen soluciones como **Apache Spark MLlib**, una de las bibliotecas más utilizadas para la implementación de algoritmos de **Machine Learning** de forma distribuida y escalable.

¿Qué es MLlib?

MLlib es el módulo de aprendizaje automático de Apache Spark, diseñado para proporcionar herramientas de modelado predictivo capaces de manejar grandes volúmenes de datos. Su principal fortaleza radica en su integración nativa con el ecosistema de Spark, permitiendo ejecutar algoritmos de Machine Learning sobre clusters distribuidos con un rendimiento eficiente.

A través de MLlib, es posible aplicar técnicas supervisadas y no supervisadas como clasificación, regresión, clustering y reducción de dimensionalidad. Además, proporciona utilidades complementarias para la preparación de datos, evaluación de modelos y optimización de hiperparámetros.

Propósito de este manual

Introducir conceptos fundamentales

Presentar los conceptos básicos de MLlib y su importancia en el ecosistema de Big Data.

Explorar características principales

Analizar las estructuras de datos y funcionalidades que hacen de MLlib una herramienta potente.

Revisar algoritmos soportados

Examinar los diferentes algoritmos de Machine Learning disponibles en la biblioteca.

Mostrar ejemplos prácticos

Presentar implementaciones de modelos supervisados y no supervisados adaptados a entornos de Big Data.

Introducción a Machine Learning Escalable

El Machine Learning tradicional enfrenta limitaciones significativas cuando se trata de procesar grandes volúmenes de datos. MLlib surge como respuesta a esta problemática, ofreciendo una solución escalable y distribuida que aprovecha toda la potencia de Apache Spark.

¿Qué es MLlib?

MLlib es la biblioteca de **Machine Learning** incluida en **Apache Spark**, diseñada para ofrecer un conjunto de herramientas que permiten construir, entrenar y evaluar modelos predictivos de forma distribuida y escalable. Esta biblioteca nació con el objetivo de solucionar uno de los principales problemas del aprendizaje automático tradicional: la dificultad de procesar y modelar grandes volúmenes de datos.

Arquitectura distribuida

1

Procesamiento paralelo

A diferencia de otras bibliotecas de Machine Learning que están pensadas para ser utilizadas en entornos de un solo equipo, MLlib fue concebida desde sus inicios para aprovechar la arquitectura distribuida de Spark.

2

Ejecución en clústeres

Los algoritmos de MLlib pueden ejecutarse de manera paralela en clústeres de cientos o miles de nodos, reduciendo los tiempos de procesamiento.

3

Manejo de datos masivos

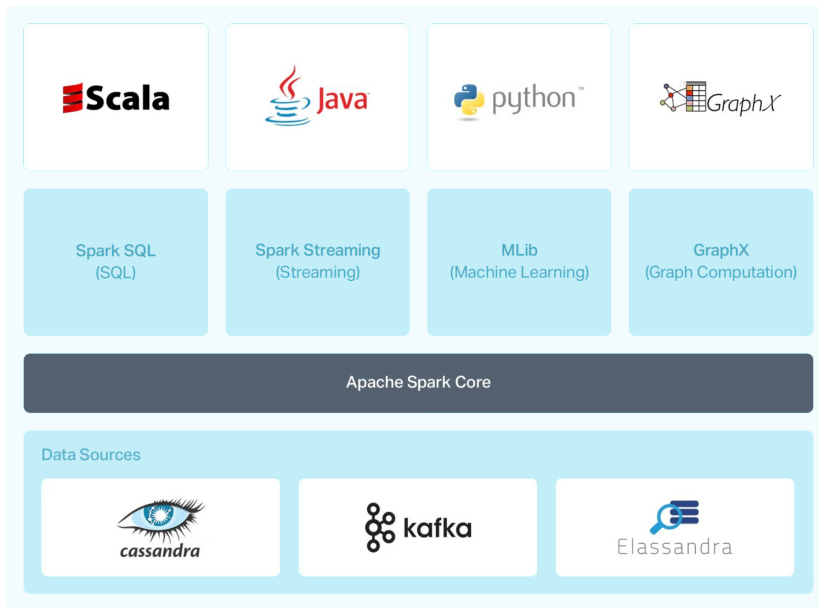
Permite trabajar con conjuntos de datos que no caben en la memoria de una sola máquina, solucionando una limitación crítica del Machine Learning tradicional.

Integración con el ecosistema Spark

Uno de los aspectos más destacados de MLlib es su integración nativa con las demás librerías y componentes de Spark, como **Spark SQL** y **Spark Streaming**. Esto permite construir flujos de trabajo completos de análisis de datos que incluyan:

- Preparación de datos
- Entrenamiento de modelos
- Predicción en tiempo real

Todo esto dentro del mismo entorno, facilitando el desarrollo de soluciones end-to-end.



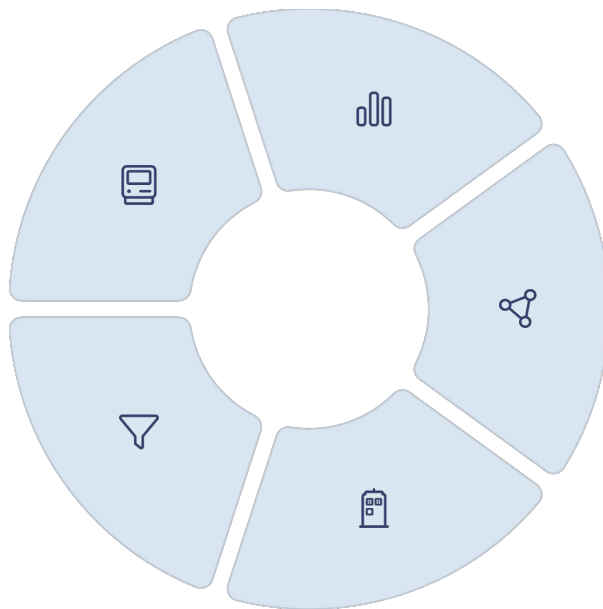
Algoritmos disponibles en MLlib

Clasificación

Algoritmos para predecir categorías o clases a partir de datos etiquetados.

Filtrado colaborativo

Algoritmos para sistemas de recomendación basados en preferencias.



Regresión

Modelos para predecir valores numéricos continuos.

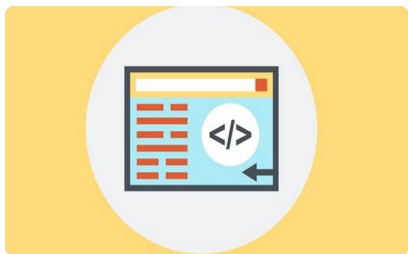
Clustering

Técnicas para agrupar datos similares sin etiquetas previas.

Reducción de dimensionalidad

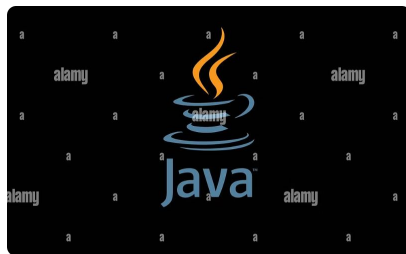
Métodos para simplificar conjuntos de datos complejos.

Soporte multilinguaje



Scala

Lenguaje nativo de Spark, ofrece la API más completa y con mejor rendimiento.



Java

Integración completa para entornos empresariales y aplicaciones existentes.



Python (PySpark)

Popular entre científicos de datos por su facilidad de uso y ecosistema de análisis.



R

Soporte para estadísticos y analistas familiarizados con este lenguaje.

Evolución de MLlib

Primeras versiones

Inicialmente basada en RDDs (Resilient Distributed Datasets), la estructura de datos fundamental de Spark.

1

2

Spark 2.0

El desarrollo se enfocó en la API basada en **DataFrames**, dejando de lado la API basada en **RDDs**.

API moderna

La nueva API proporciona un rendimiento superior y una interfaz más sencilla y expresiva para los usuarios.

3

4

MLlib Tools

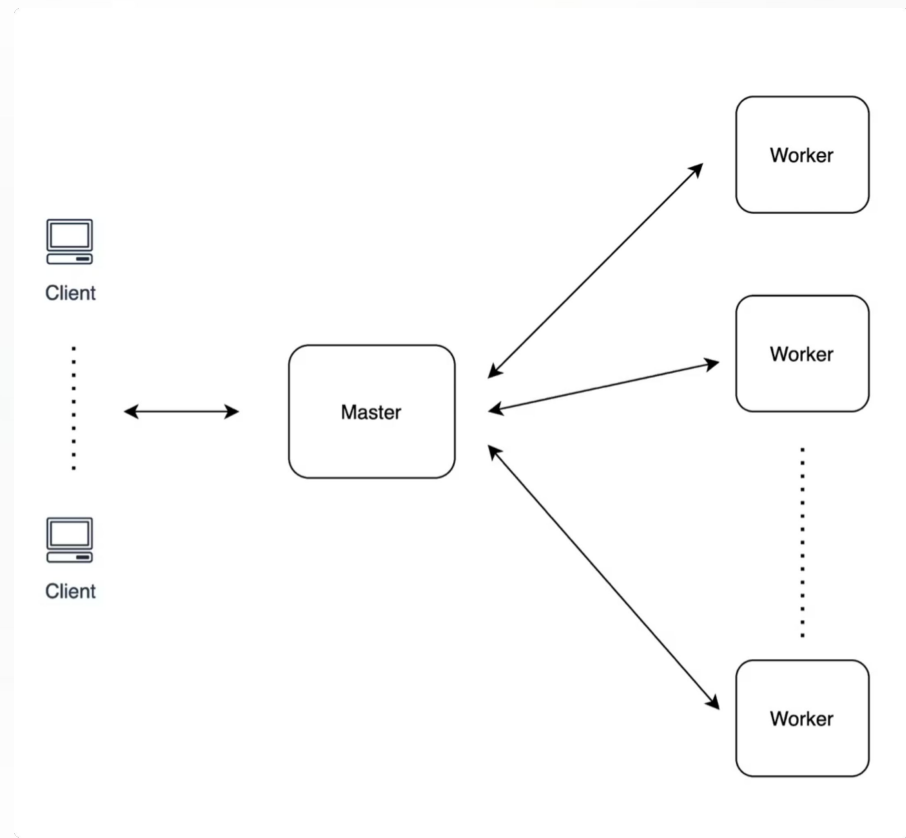
Incorporación de utilidades adicionales que facilitan tareas comunes como la carga y transformación de datos, la selección de características y la evaluación de modelos.

Resumen de MLlib

- ❗ MLlib es una biblioteca robusta y escalable que permite implementar soluciones de Machine Learning sobre grandes volúmenes de datos, integrándose de forma natural con el ecosistema Apache Spark y ofreciendo un rendimiento óptimo en entornos distribuidos.

Características de MLlib y MLlib Tools

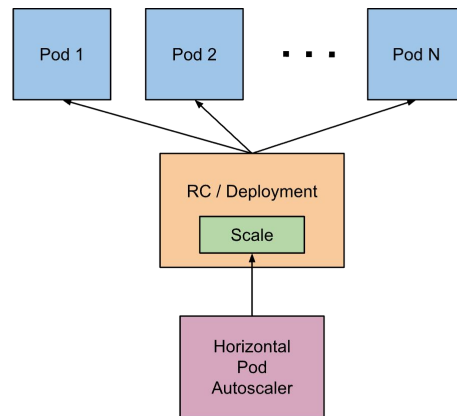
La biblioteca **MLlib** de Apache Spark destaca por ofrecer un conjunto de características que la convierten en una de las herramientas más potentes para el desarrollo de modelos de Machine Learning sobre grandes volúmenes de datos. Sus ventajas permiten a los profesionales de datos afrontar los retos de la escalabilidad y el rendimiento, esenciales en contextos de Big Data.



Escalabilidad horizontal

Una de las principales características de MLlib es su **escalabilidad horizontal**. Gracias a la arquitectura distribuida de Spark, MLlib permite:

- Procesar y modelar datasets que superan la capacidad de memoria de un solo equipo
- Ejecutar algoritmos en paralelo sobre un clúster de nodos
- Reducir considerablemente los tiempos de entrenamiento y evaluación de modelos



Compatibilidad con múltiples lenguajes

Otra característica clave es la **compatibilidad con múltiples lenguajes**. MLlib ofrece APIs en **Scala, Java, Python (PySpark) y R**, lo que facilita:

Integración flexible

Incorporación de sus funcionalidades en proyectos existentes independientemente del lenguaje utilizado.

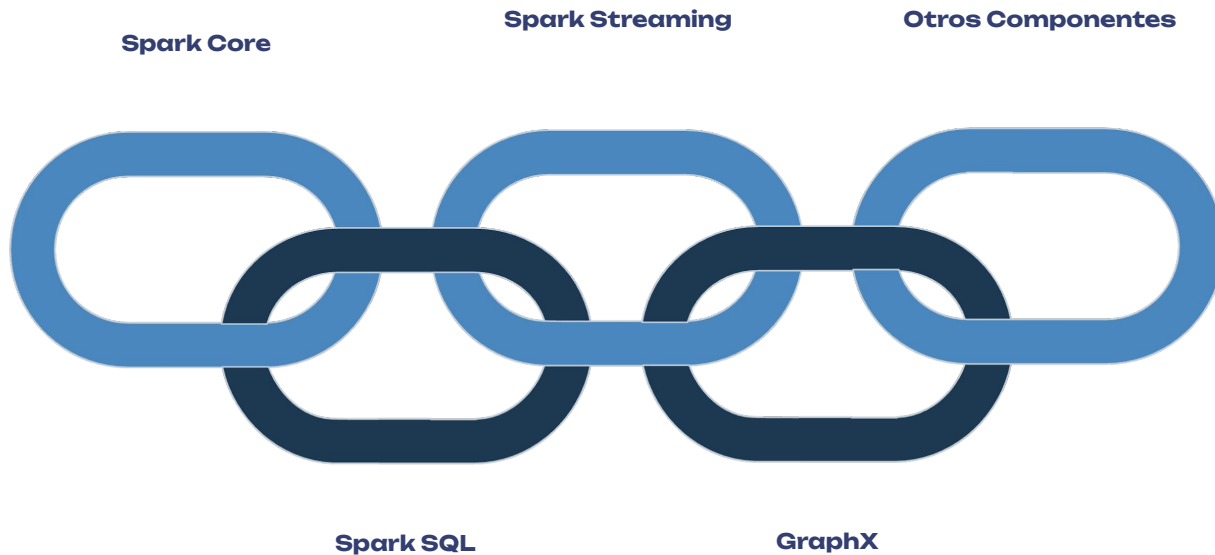
Colaboración multidisciplinaria

Facilita el trabajo entre equipos con diferentes preferencias de lenguajes de programación.

Curva de aprendizaje reducida

Los profesionales pueden utilizar el lenguaje con el que están más familiarizados.

Integración con el ecosistema Spark



MLlib se integra de forma nativa con otros módulos del ecosistema Spark, como **Spark SQL**, **Spark Streaming** y **GraphX**. Esto permite construir flujos de trabajo completos que incluyen la ingestión, transformación, análisis y visualización de datos, utilizando una única plataforma.

Catálogo de algoritmos

Aprendizaje supervisado

- Clasificación binaria y multiclase
- Regresión lineal y no lineal
- Árboles de decisión y ensambles

Aprendizaje no supervisado

- Clustering (K-means, GMM)
- Reducción de dimensionalidad (PCA)
- Detección de anomalías

Otros algoritmos

- Filtrado colaborativo
- Sistemas de recomendación
- Análisis de texto y NLP

Pipelines de Machine Learning

Una característica distintiva de MLlib es la utilización de **pipelines de Machine Learning**. Estos pipelines permiten encadenar diferentes etapas de procesamiento de datos y modelado, facilitando la automatización y reproducibilidad de los flujos de trabajo.

MLlib Tools: Transformación y preparación de datos



Normalización

Ajuste de valores numéricos a escalas comparables para mejorar el rendimiento de los algoritmos.



Escalado

Transformación de variables para que tengan propiedades estadísticas específicas.



Codificación

Conversión de variables categóricas a formatos numéricos interpretables por los algoritmos.



Selección de características

Identificación de las variables más relevantes para el modelo, reduciendo dimensionalidad.

MLlib Tools: Evaluación de modelos

Métricas para clasificación

- Precisión (Precision)
- Exhaustividad (Recall)
- F1-score
- Área bajo la curva ROC

Métricas para regresión

- Error cuadrático medio (MSE)
- Raíz del error cuadrático medio (RMSE)
- Error absoluto medio (MAE)
- R^2 (coeficiente de determinación)

Estas métricas permiten evaluar objetivamente el rendimiento de los modelos y comparar diferentes aproximaciones.

MLlib Tools: Optimización de hiperparámetros



Definición de parámetros

Identificación de los hiperparámetros a optimizar y sus rangos de valores posibles.



Grid Search

Búsqueda exhaustiva en una cuadrícula de combinaciones de parámetros predefinidas.



Validación cruzada

Evaluación de cada combinación mediante particiones múltiples de los datos.



Selección del mejor modelo

Identificación de la combinación de parámetros que maximiza el rendimiento según las métricas elegidas.

Compatibilidad con formatos de datos

Otra ventaja de MLlib es su compatibilidad con los formatos de datos utilizados en Spark, como **DataFrames** y **Datasets**. Esto facilita la manipulación y transformación de los datos antes y después del modelado.

CHANGING DATA TYPE A GUIDE TO TRANSFORMING DATA FORMATS

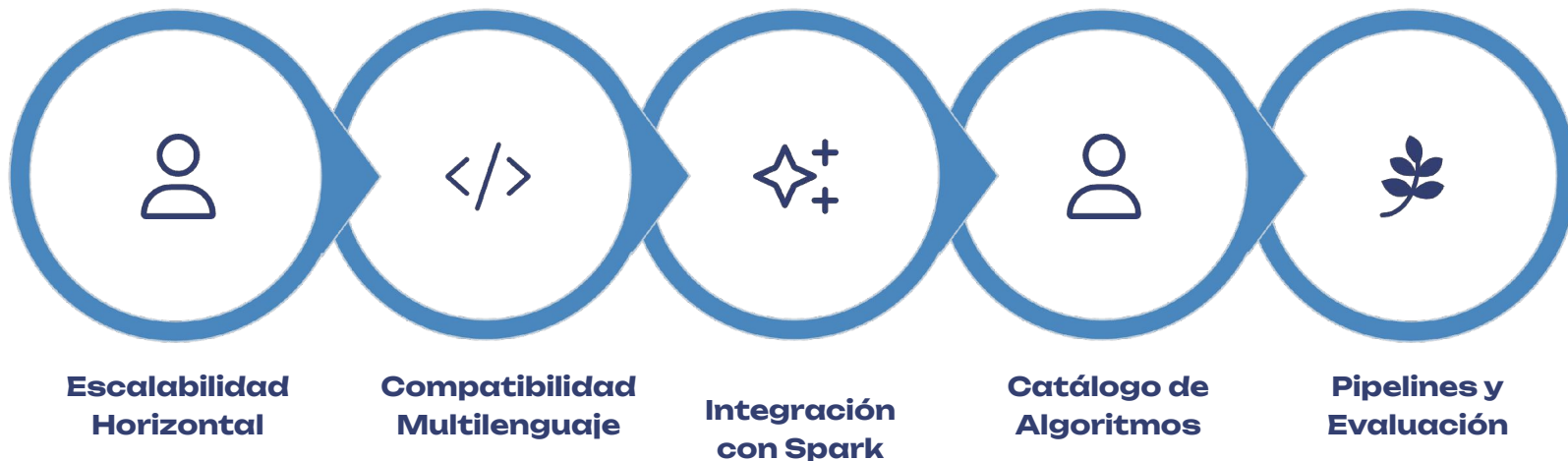
Data Type	Description
Integer	A whole number without decimal places.
Float	A number with decimal places.
String	A sequence of characters.
Boolean	Represents either true or false.
Date	Represents a specific calendar date.
Time	Represents a specific time of day.
DateTime	Represents a specific date and time.
Char	A fixed-length string.
Varchar	A variable-length string.
Text	Stores large amounts of text data.
Blob	Stores binary data, such as images or files.
Decimal	A precise decimal number with a fixed scale.
Array	A collection of values of the same data type.
JSON	Stores structured data in a human-readable format.
Enum	A set of predefined values for a column.
Object	Stores complex, unstructured data.
XML	Stores hierarchical data in a text format.
Geometric	Stores geometric shapes, such as points or polygons.
UUID	Universally unique identifier.
Interval	Represents a duration of time.
Money	Stores currency values.
Percentage	Represents a fraction out of 100.

Evolución continua

- ❏ MLlib está en constante evolución y es mantenida por la comunidad Apache Spark. Cada nueva versión incorpora mejoras de rendimiento, nuevos algoritmos y mayor compatibilidad con los estándares de la industria.

Esta evolución garantiza que MLlib se mantenga relevante y actualizada con las últimas técnicas y avances en el campo del Machine Learning.

Resumen de características



En resumen, las características de MLlib y MLlib Tools hacen de esta biblioteca una solución robusta, escalable y eficiente para implementar proyectos de Machine Learning en entornos de Big Data.

Estructuras de datos de MLlib

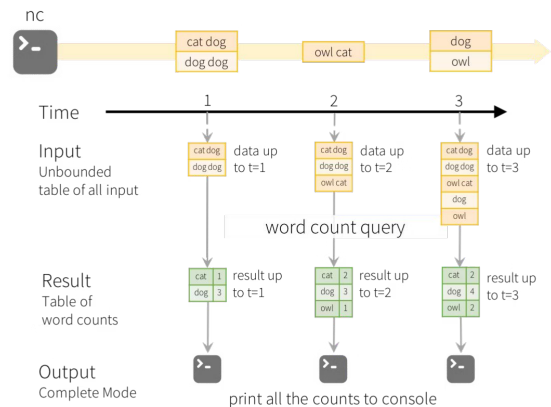
Para operar eficientemente sobre grandes volúmenes de datos, **MLlib** utiliza estructuras de datos optimizadas y adaptadas al procesamiento distribuido. Estas estructuras permiten manejar información de forma eficiente, tanto para la preparación de los datos como para la ejecución de los algoritmos de Machine Learning.

DataFrame: estructura principal

La principal estructura de datos utilizada por MLlib es el **DataFrame**. Un DataFrame en Spark es una colección distribuida de datos organizados en columnas, similar a una tabla en una base de datos relacional.

Los DataFrames ofrecen:

- Operaciones eficientes de filtrado, agrupamiento y transformación
- Escalabilidad para grandes volúmenes de datos
- API de alto nivel



Model of the Quick Example

Uso de DataFrames en MLlib

Datos de entrada

Los DataFrames contienen los datos de entrada para los modelos de Machine Learning, incluyendo tanto las características (features) como las etiquetas (labels) asociadas a cada registro.

Transformaciones

Permiten aplicar operaciones de preparación de datos como limpieza, normalización y codificación de variables categóricas.

Resultados

Los DataFrames son utilizados para almacenar los resultados del modelado, como las predicciones y las métricas de evaluación.

Vectores en MLlib

Otra estructura fundamental en MLlib es el **Vector**. Este tipo de dato se utiliza para representar las características numéricas de cada observación de manera compacta y eficiente.

DenseVector

Utilizado cuando la mayoría de los elementos son diferentes de cero. Almacena todos los valores en un array.

Ejemplo: [1.0, 2.0, 3.0, 4.0]

SparseVector

Utilizado cuando la mayoría de los elementos son ceros.

Almacena únicamente los índices y valores de los elementos no nulos, optimizando el uso de memoria.

Ejemplo: (10, [0,3,5], [1.0,4.0,2.0]) - Vector de tamaño 10 con valores no nulos en las posiciones 0, 3 y 5.

Matrices en MLlib

Además de los vectores, MLlib utiliza estructuras como **Matrices** para representar datos bidimensionales, especialmente en algoritmos como la descomposición en valores singulares (SVD) o la factorización de matrices para sistemas de recomendación.

Al igual que con los vectores, existen implementaciones densas y dispersas para optimizar el uso de memoria según la naturaleza de los datos.

Live Coding

¿En qué consistirá la Demo?

Exploraremos conceptualmente cómo preparar un entorno en PySpark para trabajar con MLlib y ejecutar un flujo básico de Machine Learning sobre un gran volumen de datos.

1. Crear una SparkSession con soporte para MLlib.
2. Cargar datos en un DataFrame y aplicar transformaciones iniciales.
3. Representar características como vectores y preparar datos para el modelo.
4. Ejecutar un algoritmo simple de clasificación o regresión.
5. Evaluar el modelo con métricas de MLlib Tools.



Momento:

Time-out!



5 -10 min.





Ejercicio

Explorando MLlib y sus estructuras de datos

Explorando MLlib y sus estructuras de datos

Contexto: 🙌

Queremos que te familiarices con las principales estructuras y herramientas que ofrece MLlib para el trabajo con grandes volúmenes de datos.

Consigna: 📝

1. Investiga y describe los componentes clave de MLlib y su integración con Spark.
2. Explica la diferencia entre un DenseVector y un SparseVector, y cuándo conviene usar cada uno.
3. Identifica tres algoritmos supervisados y dos no supervisados disponibles en MLlib.
4. Explica el concepto de pipeline en MLlib y su utilidad.

Paso a paso: ⚙️

1. Utiliza la documentación oficial de MLlib como referencia.
2. Resume la información en un esquema o cuadro comparativo.
3. Presenta ejemplos de casos de uso para cada algoritmo.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Comprendimos qué es MLlib y cómo se integra con Apache Spark.**
- ✓ **Identificamos algoritmos supervisados y no supervisados disponibles.**
- ✓ **Conocimos las estructuras de datos clave: DataFrames, vectores y matrices.**
- ✓ **Exploramos herramientas para transformar, evaluar y optimizar modelos.**
- ✓ **Sentamos las bases para elaborar un modelo de Machine Learning escalable con MLlib.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

